



# Web Programming For Back End - Unidade IV

Olá, estudantes!

Sejam bem-vindos(as) à **unidade IV** da disciplina **Web Programming For Back End**!

A disciplina **Web Programming for Back End** tem como objetivo capacitar os alunos no desenvolvimento de aplicações robustas e escaláveis, com foco na interação entre servidores e bancos de dados. Ao longo da disciplina, exploramos conceitos fundamentais para estruturar APIs eficientes, seguras e bem documentadas.

Nesta unidade, vamos nos aprofundar nas técnicas de consulta e recuperação de informações, garantindo que os dados sejam extraídos, processados e disponibilizados de forma organizada. Abordaremos boas práticas de otimização de queries, segurança no acesso a dados e a construção de endpoints que conectam aplicações ao banco de dados de maneira ágil e confiável.

Essa é uma habilidade essencial para desenvolvedores back-end que desejam criar sistemas dinâmicos e integráveis a diferentes plataformas. Vamos juntos transformar dados em soluções inteligentes! 🚀

Vamos nessa!

 Aula 1: Implementando um Banco de Dados para gerar uma API - Parte I

 Aula 2: Implementando um Banco de Dados para gerar uma API - Parte II

 Download do Conteúdo

# Aula 1: Implementando um Banco de Dados para gerar uma API - Parte I

---

## Videoaula 21: Vamos aprender a como interagir com Banco de Dados?

Os bancos de dados são fundamentais para armazenar e gerenciar informações de forma eficiente. Nesta videoaula, você aprenderá os conceitos essenciais para interagir com um banco de dados, explorando comandos básicos de consulta, inserção, atualização e remoção de dados. Com uma abordagem prática, vamos mostrar como conectar sua aplicação a um banco de dados e realizar operações que garantem o funcionamento dinâmico de sistemas e APIs.

Aperte o play e confira!



## QUAL É A IMPORTÂNCIA PARA SUA ATUAÇÃO PROFISSIONAL? —

Conhecer as dependências para conectar e interagir com banco de dados é uma questão fundamental para a devida manipulação de dados que as aplicações Back-end precisam gerar e trabalhar.

Na atualidade temos dois tipos de bibliotecas para fazer esse tipo de interação através do node.JS, são elas o Sequelize e o Prisma.

# Acessando Banco de Dados através do Node.JS

## ORM Sequelize

O Sequelize é um ORM (Object-Relational Mapping) para Node.js, que facilita a comunicação com bancos de dados SQL. Ele permite manipular dados usando JavaScript em vez de escrever queries SQL diretamente.

**Compatível com diversos bancos de dados:**

- PostgreSQL
- MySQL
- MariaDB
- SQLite
- Microsoft SQL Server

## ORM Prisma

O Prisma é um ORM (Object-Relational Mapping) para Node.js e TypeScript, que facilita a interação com bancos de dados SQL e

NoSQL. Ele serve para simplificar o gerenciamento de dados, permitindo que você consulte, crie, atualize e exclua registros do banco de maneira mais intuitiva e segura.

## **Principais Finalidades do Prisma**

### **1. Facilitar a Comunicação com Bancos de Dados**

- Substitui consultas SQL manuais por uma API fluida e orientada a objetos.
- Reduz erros comuns ao trabalhar diretamente com SQL.

### **2. Automatizar a Geração e Gerenciamento de Esquemas**

- O Prisma permite definir tabelas e relações no arquivo `schema.prisma`.
- Ele gera as migrações automaticamente para o banco de dados.

### **3. Aumentar a Produtividade e Segurança**

- Reduz código repetitivo em comparação com SQL bruto.
- Garante tipagem forte ao utilizar TypeScript.
- Ajuda a prevenir SQL Injection e outros ataques comuns.

## 4. Compatível com Diversos Bancos de Dados

- PostgreSQL
- MySQL
- SQLite
- SQL Server
- MongoDB

## 5. Instalação e configuração do ORM Prisma

Instalação do ORM

```
npm install prisma --save-dev
```

```
npx prisma init
```

Configuração do Banco de DadosNo arquivo **.env**, defina a URL do banco de dados:

```
DATABASE_URL="mysql://usuario:senha@localhost:3306/meubanco"
```

No arquivo **schema.prisma**, configure o banco desejado:

```
generator client {  
  
  provider = "prisma-client-js"  
  
}  
  
datasource db {  
  
  provider = "mysql" // Altere conforme o banco (mysql, postgresql,  
sqlite, etc.)  
  
  url      = env("DATABASE_URL")  
  
}
```

Se precisar pode utilizar o Prisma para gerenciar o seu banco de dados e criar as suas tabelas de forma automática:

Ex:

```
model User {  
  
  id    Int    @id @default(autoincrement()) // Chave primária  
  
  name  String  
  
  email String @unique  
  
  password String  
  
  createdAt DateTime @default(now())
```



```
}
```

Gerando as tabelas e sincronizando com o banco de dados

```
npx prisma migrate dev --name init
```

## 6. Usando o Prisma client

Instalando o Prisma Client

```
npm install @prisma/client
```

Criando uma instância do Prisma Client

```
import { PrismaClient } from '@prisma/client';  
  
const prisma = new PrismaClient();
```

**Videoaula 22: Implementando a busca de dados provenientes de um Banco de Dados**

## para gerar uma API

A busca de dados é um dos pilares de qualquer API eficiente. Nesta videoaula, você verá como recuperar informações de um banco de dados e estruturá-las para serem consumidas por diferentes aplicações. Vamos explorar consultas otimizadas, boas práticas de desenvolvimento e como transformar os dados em respostas organizadas e seguras para sua API.



### QUAL É A IMPORTÂNCIA PARA SUA ATUAÇÃO PROFISSIONAL? —

Além de manipular dados de um banco de dados para a criação de uma API, é fundamental que o desenvolvedor tenha habilidades sobre a manipulação de dados com ARRAY e JSON para assim conseguir trabalhar com a criação da sua API.

# Manipulando dados em Objetos

## Javascript Array e JSON

1

### Array

Manipular arrays no Node.js funciona da mesma forma que no JavaScript padrão, pois o Node.js utiliza o motor V8 do Chrome.

#### 1. Criando Arrays

```
let arr1 = [1, 2, 3, 4, 5];
```

```
let arr2 = new Array(5); // Cria um array com 5 posições vazias
```

```
let arr3 = Array.of(1, 2, 3); // Cria um array com os valores fornecidos
```

```
let arr4 = Array.from("Hello"); // Cria um array a partir de um iterável
```

## 2. Acessando Elementos

```
console.log(arr1[0]); // Primeiro elemento: 1
```

```
console.log(arr1[arr1.length - 1]); // Último elemento: 5
```

## 3. Adicionando e Removendo Elementos

### Adicionar no Final (push)

```
arr1.push(6);
```

```
console.log(arr1); // [1, 2, 3, 4, 5, 6]
```

### Remover do Final (pop)

```
arr1.pop();  
  
console.log(arr1); // [1, 2, 3, 4, 5]
```

### Adicionar no Início (unshift)

```
arr1.unshift(0);  
  
console.log(arr1); // [0, 1, 2, 3, 4, 5]
```

### Remover do Início (shift)

```
arr1.shift();  
  
console.log(arr1); // [1, 2, 3, 4, 5]
```

## 4. Métodos de Iteração

### forEach()

Executa uma função para cada elemento do array.

```
arr1.forEach((item, index) => {  
  
  console.log(`Index: ${index}, Valor: ${item}`);  
  
});
```

### map()

Cria um novo array aplicando uma função a cada elemento.

```
let arrDobro = arr1.map(num => num * 2);  
  
console.log(arrDobro); // [2, 4, 6, 8, 10]
```

## **filter()**

Cria um novo array com elementos que passam em um teste.

```
let pares = arr1.filter(num => num % 2 === 0);  
  
console.log(pares); // [2, 4]
```

## **reduce()**

Reduz um array a um único valor.

```
let soma = arr1.reduce((acumulador, valor) => acumulador +  
  valor, 0);  
  
console.log(soma); // 15
```

## **some()**

Verifica se pelo menos um elemento passa em um teste.

```
let temPar = arr1.some(num => num % 2 === 0);  
  
console.log(temPar); // true
```

## **every()**

Verifica se todos os elementos passam em um teste.

```
let todosPositivos = arr1.every(num => num > 0);  
  
console.log(todosPositivos); // true
```

## **5. Encontrar e Manipular Elementos**

### **find()**

Retorna o primeiro elemento que satisfaz um teste.



```
let primeiroMaiorQue3 = arr1.find(num => num > 3);  
  
console.log(primeiroMaiorQue3); // 4
```

## **findIndex()**

Retorna o índice do primeiro elemento que satisfaz um teste.

```
let indexMaiorQue3 = arr1.findIndex(num => num > 3);  
  
console.log(indexMaiorQue3); // 3
```

## **includes()**

Verifica se um valor está presente no array.

```
console.log(arr1.includes(3)); // true  
  
console.log(arr1.includes(10)); // false
```

Em Node.js, um objeto JSON é basicamente um objeto JavaScript.

```
let pessoa = {  
  
  nome: "João",  
  
  idade: 30,  
  
  endereco: {  
  
    rua: "Rua A",  
  
    cidade: "São Paulo"  
  
  },  
  
  hobbies: ["futebol", "música"],  
  
  ativo: true  
  
};  
  
// Acessando propriedades
```

```
console.log(pessoa.nome); // "João"

console.log(pessoa["idade"]); // 30

console.log(pessoa.endereco.cidade); // "São Paulo"

console.log(pessoa.hobbies[1]); // "música"
```

## Convertendo Objetos para JSON e JSON para Objetos

**JSON.stringify()** – Converte um objeto para uma string JSON

```
let jsonString = JSON.stringify(pessoa);

console.log(jsonString);

//    {"nome":"João","idade":30,"endereco":{"rua":"Rua
A","cidade":"São Paulo"},"hobbies":
["futebol","música"],"ativo":true}
```

**JSON.parse()** – Converte uma string JSON para um objeto JavaScript

```
let jsonObjeto = JSON.parse(jsonString);
```

```
console.log(jsonObjeto.nome); // "João"
```

## Manipulando Arrays de Objetos JSON

Muitas vezes trabalhamos com listas de objetos JSON.

```
let usuarios = [  
  
  { id: 1, nome: "Ana", idade: 25 },  
  
  { id: 2, nome: "Carlos", idade: 32 },  
  
  { id: 3, nome: "Beatriz", idade: 28 }  
  
];
```

## Filtrando elementos (filter)

```
let maioresDe30 = usuarios.filter(usuario => usuario.idade >  
30);  
  
console.log(maioresDe30); // [{ id: 2, nome: "Carlos",  
idade: 32 }]
```

## Encontrando um elemento (find)

```
let usuarioEncontrado = usuarios.find(usuario =>
  usuario.nome === "Beatriz");

console.log(usuarioEncontrado); // { id: 3, nome: "Beatriz",
  idade: 28 }
```

## Atualizando um elemento (map)

```
let usuariosAtualizados = usuarios.map(usuario =>

  usuario.id === 2 ? { ...usuario, idade: 33 } : usuario

);

console.log(usuariosAtualizados);
```

# Guia prático de TypeScript

Melhore suas aplicações JavaScript



 Casa do  
Código | alura

THIAGO DA SILVA ADRIANO

Capa do livro

## Guia prático de TypeScript: melhore suas aplicações JavaScript

O livro *Guia Prático de TypeScript: Melhore suas Aplicações JavaScript* é um excelente recurso para quem deseja aprender ou aprimorar suas habilidades com essa poderosa linguagem.

Com uma abordagem didática e exemplos práticos, o livro ensina como utilizar **tipagem estática, interfaces, classes, módulos e outras funcionalidades avançadas** para escrever códigos mais robustos e fáceis de manter. Além disso, explora boas práticas e integração com frameworks

populares, tornando-se um guia indispensável tanto para iniciantes quanto para programadores experientes.

Se você deseja elevar o nível das suas aplicações JavaScript e tornar seu código mais previsível e escalável, esta leitura é altamente recomendada! 📖🚀

**Boa Leitura** 😊📖

#### **Acesso ao Livro na Biblioteca Virtual UniFECAF**

Para acessar o livro e realizar a leitura clique no botão ao lado.

**ACESSAR O LIVRO**

## **Saiba mais**

Para trabalhar criando API é fundamental que o profissional possa dominar a lógica de programação no que se diz respeito a ARRAY e JSON, por isso nesse material o professor aborda todos os conhecimentos fundamentais que o aluno precisa compreender para conseguir utilizar esses dois assuntos tão importantes no processo de criação de API.



---

Prepare-se para mais uma jornada de conhecimento 🧐

Nos vemos na próxima aula!

**CLIQUE AQUI**



## Aula 2: Implementando um Banco de Dados para gerar uma API - Parte II

---

### Videoaula 23: Criação da API de livros para buscar e excluir dados

Nesta videoaula, daremos mais um passo na construção de uma API funcional. Você aprenderá a implementar a busca de livros no banco de dados, garantindo eficiência na recuperação das informações. Além disso, veremos como estruturar a exclusão de dados de maneira segura e controlada, seguindo boas práticas para evitar falhas e impactos indesejados no sistema.

Aperte o play e confira!



#### QUAIS HABILIDADES SERÃO DESENVOLVIDAS A PARTIR DESSES TÓPICOS? —

O JWT é uma forma eficiente de autenticar usuários sem precisar manter sessões no servidor. No entanto, ele deve ser utilizado com boas práticas para evitar falhas de segurança. A sua utilização é fundamental no processo de desenvolvimento de uma API com as devidas regras de segurança.

# Introdução a JWT

JSON Web Token (JWT) é um padrão aberto (RFC 7519) usado para transmitir informações de forma segura entre partes como um objeto JSON. Ele é amplamente utilizado para autenticação e autorização em aplicações web e APIs.

## Como funciona o JWT?

AUTENTICAÇÃO:

ENVIO DO TOKEN:

VALIDAÇÃO DO TOKEN:

- O usuário faz login enviando suas credenciais (e-mail e senha) para o servidor.
- O servidor valida as credenciais e, se estiverem corretas, gera um token JWT assinado e o retorna ao cliente.

AUTENTICAÇÃO:

ENVIO DO TOKEN:

VALIDAÇÃO DO TOKEN:

- O cliente (navegador ou aplicativo) armazena esse token (geralmente no Local Storage, Session Storage ou em um Cookie HTTP).
- Em cada requisição subsequente à API protegida, o cliente envia o token no cabeçalho HTTP (Authorization: Bearer TOKEN\_AQUI).

AUTENTICAÇÃO:

ENVIO DO TOKEN:

VALIDAÇÃO DO TOKEN:

- O servidor recebe a requisição, extrai o token e verifica se é válido (assinatura e prazo de expiração).
- Se for válido, permite o acesso ao recurso protegido; caso contrário, retorna erro de autenticação.

## **Estrutura de um JWT**

O JWT é composto por três partes separadas por pontos (.):

HEADER.PAYLOAD.SIGNATURE

- Header (Cabeçalho)

Contém o tipo do token (JWT) e o algoritmo de assinatura (ex: HS256):

```
{  
  
  "alg": "HS256",  
  
  "typ": "JWT"  
}
```

- Payload (Corpo da mensagem)

Contém as informações (claims) do usuário, como ID e permissões:

```
{
```

```
"sub": "1234567890",  
  
"name": "João Silva",  
  
"iat": 1710000000,  
  
"exp": 1710600000  
  
}
```

1. sub: Identificação do usuário.
2. iat: Data de emissão.
3. exp: Data de expiração do token.

- **Signature (Assinatura)**

A assinatura é gerada com um segredo e um algoritmo seguro (ex: HMAC SHA256):

```
HMACSHA256(  
  
base64UrlEncode(header) + "." +  
  
base64UrlEncode(payload),  
  
segredo
```

)

## Instalação da dependência JWT

```
npm install express jsonwebtoken
```

Exemplo de servidor Express e gerando o Token

```
const express = require("express");

const jwt = require("jsonwebtoken");

const app = express();

app.use(express.json());

const SECRET_KEY = process.env.SECRET_KEY ||
"chave_super_secret";

// Rota de login

app.post("/login", (req, res) => {

  const { username } = req.body;

  if (!username) {
```

```
    return res.status(400).json({ message: "Usuário é obrigatório" });  
  
  }  
  
  // Criando payload do JWT  
  
  const payload = { username };  
  
  // Gerando token com expiração de 1 hora  
  
  const token = jwt.sign(payload, SECRET_KEY, { expiresIn: "1h" });  
  
  res.json({ token });  
  
});
```

## Videoaula 24: Criação da API de livros para atualizar dados

Atualizar informações é uma funcionalidade essencial em qualquer API. Nesta videoaula, você aprenderá como implementar a edição de registros em um banco de dados dentro da



API de livros. Vamos abordar desde a validação dos dados até a execução das operações de atualização, garantindo integridade e segurança para a aplicação.

Dê o play e confira!



**QUAIS HABILIDADES SERÃO DESENVOLVIDAS A PARTIR DESSES TÓPICOS?**

O TypeScript melhora a segurança e manutenção do código, tornando-o mais escalável. Ele adiciona tipagem estática, classes, interfaces e muitas funcionalidades úteis sem perder a compatibilidade com JavaScript.

# O que é TypeScript?

TypeScript é um superset do JavaScript que adiciona tipagem estática e recursos modernos à linguagem. Ele foi desenvolvido pela Microsoft para ajudar a tornar o desenvolvimento de aplicações grandes mais seguro e escalável.

- Arquivos TypeScript possuem a extensão .ts
- O TypeScript compila para JavaScript puro, o que significa que pode rodar em qualquer navegador ou ambiente que suporte JavaScript.

## Principais Vantagens do TypeScript

- **Tipagem Estática:** Reduz erros comuns antes da execução.
- **Melhor Autocompletar & IntelliSense:** Ferramentas como VSCode oferecem sugestões

melhores.

- **Refatoração mais segura:** Facilita a manutenção e evita bugs ao alterar o código.
- **Suporte a ESNext:** Permite usar recursos modernos antes do suporte total nos navegadores.
- **Orientação a Objetos:** Suporte a interfaces, classes, herança e modificadores de acesso.

## Instalação do TypeScript

Para instalar o compilador do TypeScript globalmente:

```
npm install -g typescript
```

## Tipos Primitivos no TypeScript

No TypeScript, podemos definir o tipo de uma variável, reduzindo erros.

- `let nome: string = "João";`
- `let idade: number = 25;`
- `let ativo: boolean = true;`

| Tipo      | Exemplo                                    |
|-----------|--------------------------------------------|
| string    | "Texto"                                    |
| number    | 42, 3.14                                   |
| boolean   | true, false                                |
| any       | Qualquer tipo (string, number...)          |
| void      | Indica que uma função não retorna nada     |
| null      | null                                       |
| undefined | undefined                                  |
| never     | Para funções que nunca retornam (ex: erro) |
| unknown   | Semelhante a any, mas mais seguro          |

## Inferência de Tipos

O TypeScript pode inferir o tipo automaticamente, evitando declarações redundantes.

```
let mensagem = "Olá, mundo!"; // TypeScript entende que é string
```

```
// mensagem = 42; ❌ Erro! Tipo esperado é string
```

## Arrays

```
let numeros: number[] = [1, 2, 3, 4];  
  
let nomes: Array<string> = ["Ana", "Bruno"];
```

## Tuplas

Tuplas definem um array com tipos específicos para cada posição.

```
let pessoa: [string, number] = ["João", 30];  
  
// pessoa = [30, "João"]; ❌ Erro!
```

## Enums

Enums ajudam a definir um conjunto de valores nomeados.

```
enum Status {  
  
  Pendente,  
  
  EmAndamento,  
  
  Concluido  
  
}  
  
let meuStatus: Status = Status.EmAndamento;  
  
console.log(meuStatus); // 1 (Enums começam do 0 por padrão)
```

## Funções com Tipos

```
function somar(a: number, b: number): number {  
  
  return a + b;  
  
}  
  
console.log(somar(5, 3)); // 8
```

## Parâmetros opcionais e valores padrão

```
function saudar(nome: string, saudacao: string = "Olá"):
string {

    return `${saudacao}, ${nome}!`;

}

console.log(saudar("Maria")); // "Olá, Maria!"
```

## Classes e Modificadores de Acesso

O TypeScript permite Programação Orientada a Objetos com classes.

```
class Pessoa {

    private nome: string;

    protected idade: number;

    public ativo: boolean;

    constructor(nome: string, idade: number, ativo: boolean) {

        this.nome = nome;

        this.idade = idade;

    }

}
```

```
this.ativo = ativo;

}

saudacao(): string {

    return `Olá, meu nome é ${this.nome}`;

}

}
```

```
const p = new Pessoa("Ana", 22, true);

console.log(p.saudacao());
```

## Modificadores de acesso:

- **public:** Acessível de qualquer lugar.
- **private:** Apenas dentro da classe.
- **protected:** Acessível dentro da classe e subclasses.

## Utilizando TypeScript com Node.js



Se você deseja usar TypeScript no Node.js:

Instale o TypeScript e os tipos do Node.js:

```
npm install -g typescript  
  
npm install --save-dev @types/node
```

Crie um arquivo tsconfig.json para configuração:

```
tsc --init
```

Compile o código TypeScript para JavaScript:

```
tsc
```

Execute o JavaScript gerado:

```
node dist/index.js
```

## Testando os aprendizados

# Que tal testar os aprendizados desta unidade? Vem comigo!

Com base nos estudos realizados nesta unidade, marque VERDADEIRO ou FALSO nas perguntas/afirmativas abaixo:

O JWT é armazenado no servidor e, a cada requisição, o servidor verifica seu estado para garantir que ele ainda seja válido.

☐ Verdadeiro

☐ Falso

SUBMIT

No Node.js, um array pode conter elementos de diferentes tipos, como números, strings e objetos.

---

☐ Verdadeiro

☐ Falso

SUBMIT

Aula Ao Vivo

---

Quantos aprendizados até aqui, não é mesmo!?

Agora, te convido a participar da aula ao vivo 🎬

Confira abaixo os temas que serão abordados, você não pode perder! 🧑🧑

|                                                                                   |                                                                           |
|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------|
|  | Implementar um CRUD com os alunos                                         |
|  | Implementar o modelo MVC para a organização e padronização da programação |
|  | Criar uma API que interage com o banco de dados                           |

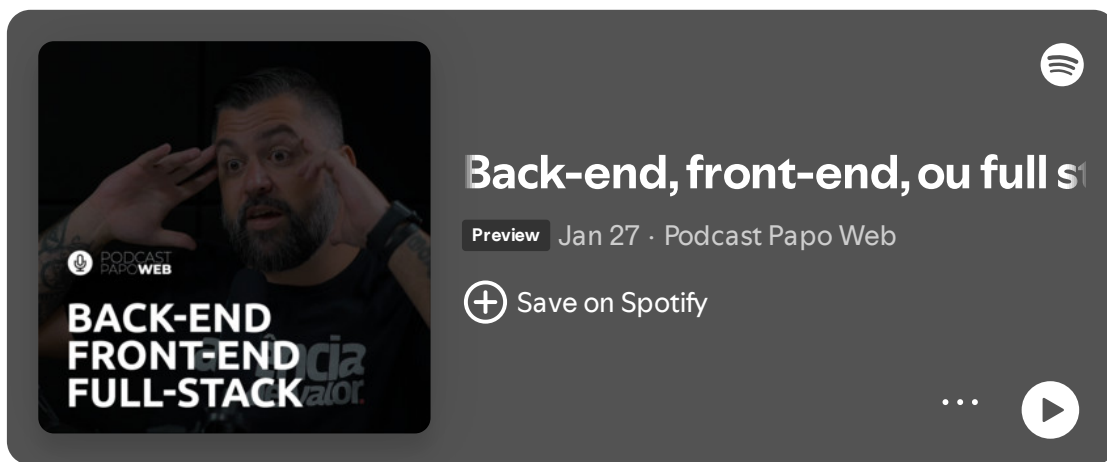


Essas atividades tornam as aulas ao vivo mais dinâmicas e produtivas, estimulando o protagonismo dos estudantes, consolidando o aprendizado e fortalecendo a reflexão crítica sobre a articulação entre teoria e prática.

## Sintonia do Saber

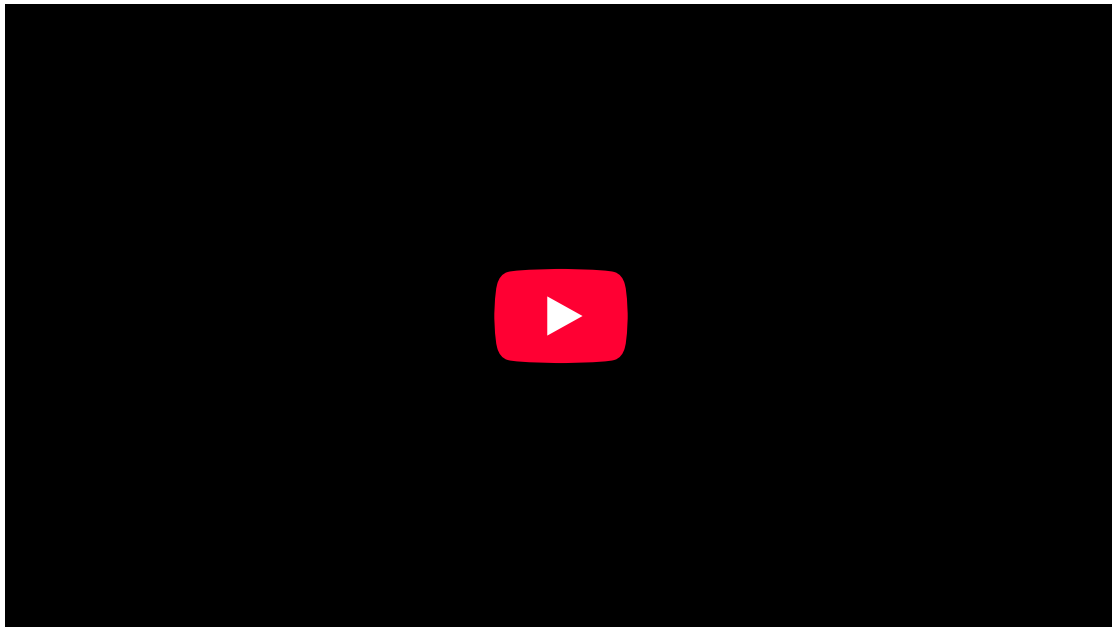
Neste material você irá entender as principais diferenças entre desenvolvedores Front-End, Back-End e Full-Stack. Neste podcast você irá compreender onde cada profissional atua em um projeto de desenvolvimento de software.

Dê o play e ouça 🎧



Saiba mais

A segurança em aplicações Back-end é um assunto muito importante no processo de criação de uma API, portanto aprender a utilizar o JWT é crucial para o desenvolvimento de aplicações modernas.



---

Parabéns, você finalizou a unidade 🧐

Faça download do conteúdo!

[CLIQUE AQUI](#)

## Download do Conteúdo

---

---

Que tal baixar todo esse conteúdo e aproveitar os estudos  
no seu melhor momento?

Faça o download do conteúdo dessa unidade:



**BAIXE O CONTEÚDO DESTA UNIDADE.pdf**

1.6 MB

